

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

* Teste Global de Época Normal (16 de Janeiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [8 valores]

Nas seguintes questões, selecione a única opção verdadeira. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

1. Um *middleware* express:
 - A. Não pode acrescentar propriedades ao objeto *response*
 - B. Não pode acrescentar propriedades ao objeto *request*
 - C. Pode interceptar um pedido antes de ele chegar ao *handler*
 - D. Nenhuma das anteriores
2. Considere o seguinte URL: <http://www.cmdb.com/movies?page=5&limit=5>
 - A. A *path* neste URL é: `page=5&limit=5`
 - B. A *query string* tem dois parâmetros *page* e *limit*
 - C. www.cmdb.com identifica um recurso
 - D. Nenhuma das anteriores
3. Uma aplicação *web* recebe um pedido que não consegue processar, uma vez que o URI não refere um recurso existente. O status code adequado para responder a esta situação é:
 - A. 200
 - B. 400
 - C. 404
 - D. 500
4. Uma aplicação *web* recebe um pedido de GET para <http://www.cmdb.com/movies/1/create>, deve:
 - A. Criar um recurso com identificador 1 e responder com o status code 201
 - B. Responder com status code 404 se o recurso não existir
 - C. Responder com status code 500 se o recurso não existir
 - D. Nenhuma das anteriores
5. Uma aplicação *web* recebe um pedido GET para <http://www.cmdb.com/movies/1>, no entanto, não consegue obter informação sobre o recurso porque não é possível estabelecer uma ligação com a base de dados. O status code adequado para responder a esta situação é:
 - A. 200
 - B. 400
 - C. 404
 - D. 500
6. Um browser recebe uma resposta HTTP com status code 303. Como consequência:
 - A. Apresenta o conteúdo HTML presente no *body*.
 - B. Faz um pedido GET para o URL indicado no *header* Content-Type
 - C. Faz um pedido GET para o URL indicado no *header* Location.
 - D. Faz um pedido do tipo definido no header Method, para o URL indicado no *header* Location.
7. Um pedido HTTP considerado *safe*:
 - A. Pode ser feito usando o método POST
 - B. Pode ser feito usando o método PUT
 - C. Pode ser feito usando qualquer dos métodos HTTP
 - D. Nenhuma das anteriores

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

* Teste Global de Época Normal (16 de Janeiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

8. Quando é feito um pedido POST com o body em JSON:
- A. Deve ser adicionado ao pedido o *header* Content com o valor application/json
 - B. Deve ser adicionado ao pedido o *header* Type com o valor application/json
 - C. Deve ser adicionado ao pedido o *header* Content-Type com o valor application/json
 - D. Nenhuma das anteriores
9. A instrução `app.use(express.json())`:
- A. Regista a função `express.json` como *middleware* de uma aplicação Express
 - B. Regista o retorno da função `express.json` como *middleware* de uma aplicação Express
 - C. Regista a função `express.json` como *handler* de uma aplicação Express.
 - D. Regista o retorno da função `express.json` como *handler* de uma aplicação Express
10. Em código JavaScript executado pelo browser no âmbito de um documento HTML, a forma correcta de obter a referência para um elemento com o identificador `user1` é:
- A. `document.querySelector("div.user1")`
 - B. `document.querySelector("id.user1")`
 - C. `document.querySelectorAll("id user1")`
 - D. `document.querySelector("#user1")`
11. Na framework express, as rotas são identificadas por:
- A. URI e método HTTP
 - B. Path do URI e método HTTP
 - C. Path query do URI e método HTTP
 - D. Query do pedido e método HTTP
12. Uma aplicação *web* recebe um pedido de DELETE para <http://www.cmdb.com/movies/1>, deve:
- A. Apagar o recurso correspondente ao URI do pedido se existir, e responder com *status code* 200 quer exista ou não.
 - B. Apagar o recurso correspondente ao URI do pedido se existir e responder com *status code* 200. Caso não exista, lançar uma exceção em JavaScript com o código 404.
 - C. Apagar o recurso correspondente ao URI do pedido se existir e responder com *status code* 200. Caso não exista, responder com *status code* 404.
 - D. Nenhuma das anteriores

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

** Teste Global de Época Normal (16 de Janeiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [8 valores]

Nas seguintes questões, selecione a única opção verdadeira. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

1. Na framework express, as rotas são identificadas por:
 - A. URI e método HTTP
 - B. Query do pedido e método HTTP
 - C. Path do URI e método HTTP
 - D. Path query do URI e método HTTP
2. Uma aplicação *web* recebe um pedido que não consegue processar, uma vez que o URI não refere um recurso existente. O status code adequado para responder a esta situação é:
 - A. 200
 - B. 400
 - C. 404
 - D. 500
3. Em código JavaScript executado pelo browser no âmbito de um documento HTML, a forma correcta de obter a referência para um elemento com o identificador user1 é:
 - A. document.querySelectorAll("id.user1")
 - B. document.querySelector("div.user1")
 - C. document.querySelectorAll("id user1")
 - D. document.querySelector("#user1")
4. Uma aplicação *web* recebe um pedido de DELETE para <http://www.cmdb.com/movies/1>, deve:
 - A. Apagar o recurso correspondente ao URI do pedido se existir, e responder com *status code* 200 quer exista ou não.
 - B. Apagar o recurso correspondente ao URI do pedido se existir e responder com *status code* 200. Caso não exista, lançar uma exceção em JavaScript com o código 404.
 - C. Apagar o recurso correspondente ao URI do pedido se existir e responder com *status code* 200. Caso não exista, responder com *status code* 404.
 - D. Nenhuma das anteriores
5. Um browser recebe uma resposta HTTP com *status code* 303. Como consequência:
 - A. Apresenta o conteúdo HTML presente no *body*.
 - B. Faz um pedido GET para o URL indicado no *header* Content-Type
 - C. Faz um pedido GET para o URL indicado no *header* Location.
 - D. Faz um pedido do tipo definido no header Method, para o URL indicado no *header* Location.
6. Quando é feito um pedido POST com o body em JSON:
 - A. Deve ser adicionado ao pedido o *header* Content com o valor application/json
 - B. Deve ser adicionado ao pedido o *header* Type com o valor application/json
 - C. Deve ser adicionado ao pedido o *header* Content-Type com o valor application/json
 - D. Nenhuma das anteriores

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

** Teste Global de Época Normal (16 de Janeiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

7. A instrução `app.use(express.json())`:
- A. Regista a função `express.json` como *middleware* de uma aplicação Express
 - B. Regista o retorno da função `express.json` como *middleware* de uma aplicação Express
 - C. Regista a função `express.json` como *handler* de uma aplicação Express.
 - D. Regista o retorno da função `express.json` como *handler* de uma aplicação Express
8. Um *middleware* express:
- A. Não pode acrescentar propriedades ao objeto `request`
 - B. Não pode acrescentar propriedades ao objeto `response`
 - C. Pode interceptar um pedido antes de ele chegar ao *handler*
 - D. Nenhuma das anteriores
9. Considere o seguinte URL: <http://www.cmdb.com/movies?page=5&limit=5>
- A. www.cmdb.com identifica um recurso
 - B. A *path* neste URL é: `page=5&limit=5`
 - C. A *query string* tem dois parâmetros `page` e `limit`
 - D. Nenhuma das anteriores
10. Uma aplicação *web* recebe um pedido GET para <http://www.cmdb.com/movies/1>, no entanto, não consegue obter informação sobre o recurso porque não é possível estabelecer uma ligação com a base de dados. O *status code* adequado para responder a esta situação é:
- A. 500
 - B. 400
 - C. 404
 - D. 200
11. Um pedido HTTP considerado *safe*:
- A. Pode ser feito usando qualquer dos métodos HTTP
 - B. Pode ser feito usando o método POST
 - C. Pode ser feito usando o método PUT
 - D. Nenhuma das anteriores
12. Uma aplicação *web* recebe um pedido de GET para <http://www.cmdb.com/movies/1/create>, deve:
- A. Responder com *status code* 404 se o recurso não existir
 - B. Criar um recurso com identificador 1 e responder com o *status code* 201
 - C. Responder com *status code* 500 se o recurso não existir
 - D. Nenhuma das anteriores

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

*** Teste Global de Época Normal (16 de Janeiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [8 valores]

Nas seguintes questões, selecione a única opção verdadeira. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

1. A instrução `app.use(express.json())`:
 - A. Regista a função `express.json` como *handler* de uma aplicação Express.
 - B. Regista a função `express.json` como *middleware* de uma aplicação Express
 - C. Regista o retorno da função `express.json` como *middleware* de uma aplicação Express
 - D. Regista o retorno da função `express.json` como *handler* de uma aplicação Express
2. Um pedido HTTP considerado *safe*:
 - A. Pode ser feito usando qualquer dos métodos HTTP
 - B. Pode ser feito usando o método POST
 - C. Pode ser feito usando o método PUT
 - D. Nenhuma das anteriores
3. Considere o seguinte URL: <http://www.cmdb.com/movies?page=5&limit=5>
 - A. A *path* neste URL é: `page=5&limit=5`
 - B. A *query string* tem dois parâmetros `page` e `limit`
 - C. www.cmdb.com identifica um recurso
 - D. Nenhuma das anteriores
4. Uma aplicação *web* recebe um pedido que não consegue processar, uma vez que o URI não refere um recurso existente. O status code adequado para responder a esta situação é:
 - A. 200
 - B. 400
 - C. 404
 - D. 500
5. Uma aplicação *web* recebe um pedido GET para <http://www.cmdb.com/movies/1>, no entanto, não consegue obter informação sobre o recurso porque não é possível estabelecer uma ligação com a base de dados. O *status code* adequado para responder a esta situação é:
 - A. 500
 - B. 400
 - C. 404
 - D. 200
6. Quando é feito um pedido POST com o body em JSON:
 - A. Deve ser adicionado ao pedido o *header* Content com o valor `application/json`
 - B. Deve ser adicionado ao pedido o *header* Type com o valor `application/json`
 - C. Deve ser adicionado ao pedido o *header* Content-Type com o valor `application/json`
 - D. Nenhuma das anteriores
7. Em código JavaScript executado pelo browser no âmbito de um documento HTML, a forma correcta de obter a referência para um elemento com o identificador `user1` é:
 - A. `document.querySelector("div.user1")`
 - B. `document.querySelectorAll("id.user1")`
 - C. `document.querySelector("#user1")`
 - D. `document.querySelectorAll("id user1")`

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

*** Teste Global de Época Normal (16 de Janeiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

8. Na framework express, as rotas são identificadas por:
- A. URI e método HTTP
 - B. Path do URI e método HTTP
 - C. Path query do URI e método HTTP
 - D. Query do pedido e método HTTP
9. Uma aplicação *web* recebe um pedido de DELETE para <http://www.cmdb.com/movies/1>, deve:
- A. Apagar o recurso correspondente ao URI do pedido se existir e responder com *status code* 200. Caso não exista, lançar uma exceção em JavaScript com o código 404.
 - B. Apagar o recurso correspondente ao URI do pedido se existir, e responder com *status code* 200 quer exista ou não.
 - C. Apagar o recurso correspondente ao URI do pedido se existir e responder com *status code* 200. Caso não exista, responder com *status code* 404.
 - D. Nenhuma das anteriores
10. Uma aplicação *web* recebe um pedido de GET para <http://www.cmdb.com/movies/1/create>, deve:
- A. Criar um recurso com identificador 1 e responder com o status code 201
 - B. Responder com status code 500 se o recurso não existir
 - C. Responder com status code 404 se o recurso não existir
 - D. Nenhuma das anteriores
11. Um browser recebe uma resposta HTTP com *status code* 303. Como consequência:
- A. Apresenta o conteúdo HTML presente no *body*.
 - B. Faz um pedido GET para o URL indicado no *header* Content-Type
 - C. Faz um pedido GET para o URL indicado no *header* Location.
 - D. Faz um pedido do tipo definido no header Method, para o URL indicado no *header* Location.

NÚMERO _____ NOME _____

GRUPO 2 [3.5 valores]

1. [1.5] Considere o programa em Node.js apresentado em seguida. Indique justificando, num máximo de 2 linhas, o que é escrito na consola.

```
console.log("BEGIN")
let running = true
setTimeout(() => { running = false }, 3000)
function wait() { while(running) console.log("waiting...") }
wait()
console.log("END")
```

2. [2] Considere o código em Node.js apresentado. Assinale nas colunas correspondentes, quais os handlers que são executados para cada um dos pedidos (Método e URI path).

```
const app = express()

app.post("/users", handler1)
app.get("/users/:id", handler2)
app.get("/users:userName", handler3)
```

	Method	URI path	Handler1	Handler2	Handler3
1	POST	/users/123/User1			
2	POST	/users/123			
3	GET	/users/123		x	
4	GET	/users/Ramos		x	
5	GET	/users?userName=Ramos			
6	POST	/users?userName=Ramos	x		

GRUPO 3 [5 valores]

1. [3] Implemente a função createAndUpdate, que a listagem seguinte mostra um exemplo da sua utilização. Esta função recebe como argumentos um *array* de objetos com a informação necessária para criar recursos através do HTTP. Cada objeto no *array* contém: o URL do recurso, o método HTTP a usar e o objeto com o conteúdo do novo recurso. Este método retorna uma Promise que quando resolvida produz um *array* com o conteúdo das respostas correspondentes ao a cada um dos pedidos.

NOTA: Valorizam-se soluções que realizem os pedidos em paralelo.

```
function main(){
  const operations = [
    {url : "http://localhost:9000/users", method : "POST", body: {userName : "userName1"}},
    {url : "http://localhost:9000/groups/1", method : "PUT", body: {name : "Group1",
description : "Description1"}}
  ]
  createAndUpdate(operations).then((resps)=>console.log("All responses received:", resps))
}
```

2. [2]. Considere o seguinte código em JavaScript.

```
1 async function f1() { return 1904 }
2 async function f2() { return "SLB" }
3 async function f3() { const num = await f1(); return await f2() + ":" + num }
4 const res = f3()
5 console.log(res)
```

- a. [0.5] O que é escrito na consola na linha 5.
b. [1.5] Implemente a função f3() sem usar *await*.

NÚMERO _____ NOME _____

GRUPO 4 [3.5 valores]

1. [3,5] Considere os módulos apresentados nas listagens seguintes, que fazem parte de uma aplicação web implementada com o módulo Express em Node.js.

service.mjs		
1	const counters = _____ // 1	15%
2		
3	function incrementPathCounter(path) {	
4	if(!counters[path])	
5	counters[path] = 2	10%
6	return _____ // 3	10%
7	}	
8		
9	function getCounter(path){ return counters[path] }	
10		
11	const service = { incrementPathCounter, getCounter }	
12		
13	export _____ // 4	10%

middleware.mjs		
1	function middlewareConstructor(service) {	
2	function middleware(req, rsp, next) {	
3	service._____ // 1	15 %
4	_____ // 2	20%
5	}	
6	return _____ // 3	20%
7	}	
8	export default middlewareConstructor	

server.mjs		
1	import express from "express";	
2	import middlewareConstructor from "../middleware.mjs"	
3	import service from "../service.mjs"	
4	const app = express()	
5	const counterMiddleware = _____ // 1	50%
6		
7	app.use(counterMiddleware)	
8	app.get("/handler1", handler1)	
9	app.get("/handler2", handler2)	
10	app._____("/counters", countersHandler) // 2	50%
11		
12	app.listen(9000, ()=> console.log("Server listening in port 1904"))	
13		
14	function handler1(req, rsp){rsp.json({message : "handler1"})}	
15	function handler2(req, rsp){rsp.json({message : "handler2"})}	
16		
17	function countersHandler(req, resp) {	
18	resp.json({	
19	url : req.path,	
20	counter : _____ // 3	
21	})	
22	}	

- a. [2] Complete a implementação dos módulos service.mjs e middleware.mjs que em conjunto, disponibilizam a funcionalidade de contagem do número de acessos aos caminhos dos URIs acedidos pelos clientes de uma aplicação Express.
- b. [1.5] Complete a implementação do módulo server, que expõe um recurso com a informação com o número de pedidos efectuados aos vários URLs.